

SQL JOINS

A Complete Beginner's Tutorial

From Zero to Confident — with Analogies, Diagrams & Practice

1. What Are SQL Joins?

In a real-world database, data is spread across multiple tables — not stuffed into one giant table. SQL Joins are the glue that connects these tables together so you can pull meaningful, combined information in a single query.

1.1 Why Do We Use Joins?

Imagine a school that keeps:

- A Students table (who the students are)
- A Courses table (what courses exist)
- An Enrollments table (which student is in which course)

Without joins, you'd have to run three separate queries and stitch the results together manually. With a JOIN, you do it in one shot.

1.2 Key Features of SQL Joins

- Combine rows from two or more tables based on a related column.
- Avoid data duplication — each piece of information lives in one place.
- Support filtering, aggregation, and complex logic across tables.
- Work with any relational database: MySQL, PostgreSQL, SQL Server, SQLite, etc.
- Multiple join types give you precise control over which rows are included.

1.3 Real-World Analogy

Real-World Situation	SQL Join Equivalent
Matching students with their report cards	Students JOIN Reports ON student_id
Linking orders with customer names	Orders JOIN Customers ON customer_id
Finding which employees work in which department	Employees JOIN Departments ON dept_id

2. Our Sample Tables

Throughout this tutorial, we will use two simple tables. Memorise them — every example uses these!

Table 1: employees

This table stores employee details.

emp_id	emp_name	dept_id
1	Alice	10
2	Bob	20
3	Charlie	30
4	Diana	10
5	Edward	NULL

Table 2: departments

This table stores department information.

dept_id	dept_name
10	HR
20	IT
40	Finance

⚠ Notice:

- Edward (emp_id=5) has NULL dept_id — not assigned to any department.
- Charlie (dept_id=30) points to a department that does NOT exist in departments.
- Finance (dept_id=40) has no employees assigned to it.

These edge cases will help us understand how different joins behave!

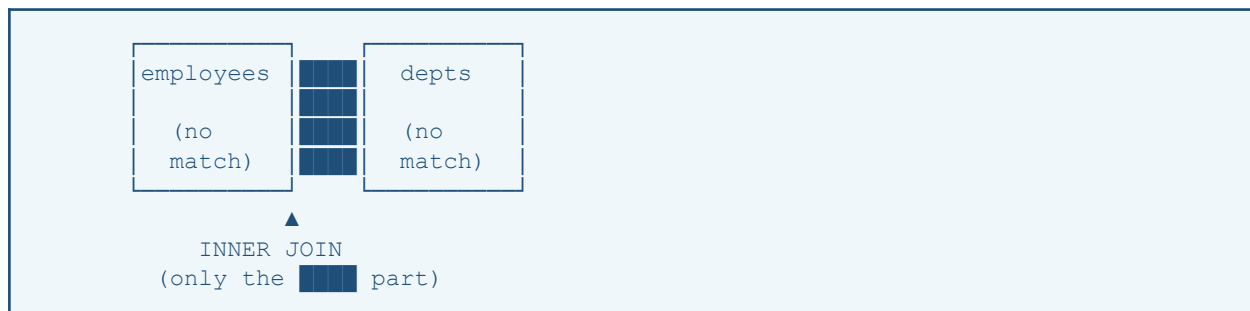
3. INNER JOIN

3.1 Concept

INNER JOIN returns only the rows where there is a MATCH in BOTH tables. Think of it as the intersection — only common data.

🔑 Real-world analogy: You have a guest list (employees) and a seating chart (departments). INNER JOIN gives you only guests who have a seat — no seat = excluded, empty seat = excluded.

3.2 Venn Diagram



3.3 SQL Syntax

```
SELECT e.emp_name, d.dept_name
FROM   employees AS e
INNER JOIN departments AS d
      ON e.dept_id = d.dept_id;
```

Explanation:

- FROM employees AS e — start with the employees table, alias it 'e'.
- INNER JOIN departments AS d — join with departments, alias 'd'.
- ON e.dept_id = d.dept_id — the matching condition: dept_id must be equal in both tables.

3.4 Expected Output

emp_name	dept_name
Alice	HR
Bob	IT
Diana	HR

3.5 Step-by-Step: How the Output is Formed

1. SQL looks at each row in employees and tries to find a matching dept_id in departments.
2. Alice → dept_id=10 → matches HR ✓ → included.
3. Bob → dept_id=20 → matches IT ✓ → included.
4. Charlie → dept_id=30 → NO match in departments ✗ → excluded.
5. Diana → dept_id=10 → matches HR ✓ → included.
6. Edward → dept_id=NULL → NULL never matches anything ✗ → excluded.
7. Finance (dept_id=40) → no employee points to it ✗ → excluded.

Result: 3 rows where both sides agree.

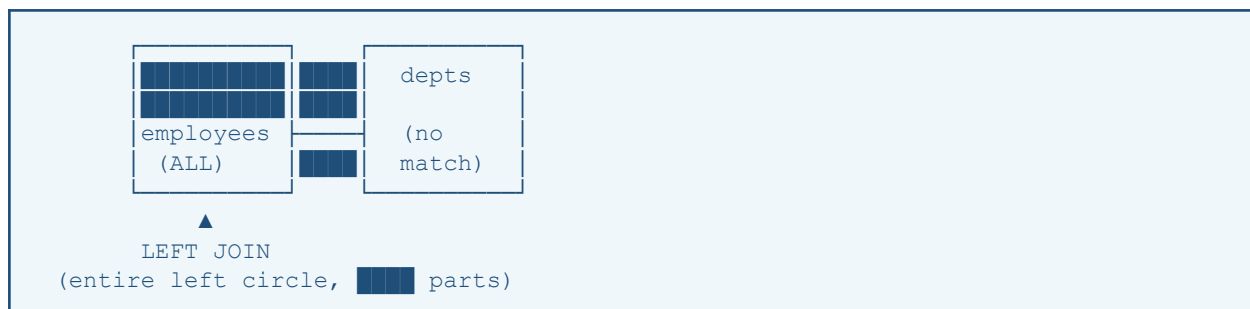
4. LEFT JOIN (LEFT OUTER JOIN)

4.1 Concept

LEFT JOIN returns ALL rows from the LEFT table, and the matched rows from the right table. If there is no match, the right side columns show NULL.

Think of it as: 'Give me everyone from the left table, and fill in any matching info from the right — if no match, leave it blank.'

4.2 Venn Diagram



4.3 SQL Syntax

```
SELECT e.emp_name, d.dept_name
FROM   employees AS e
LEFT JOIN departments AS d
      ON e.dept_id = d.dept_id;
```

4.4 Expected Output

emp_name	dept_name
----------	-----------

Alice	HR
Bob	IT
Charlie	NULL
Diana	HR
Edward	NULL

4.5 Step-by-Step Explanation

8. Every employee is included — no matter what.
9. Alice → dept_id=10 → HR ✓
10. Bob → dept_id=20 → IT ✓
11. Charlie → dept_id=30 → no dept found → dept_name = NULL.
12. Diana → dept_id=10 → HR ✓
13. Edward → dept_id=NULL → no dept found → dept_name = NULL.
14. Finance (dept_id=40) does NOT appear — it's on the right side with no match on left.

Key Insight: LEFT JOIN never loses rows from the left table.

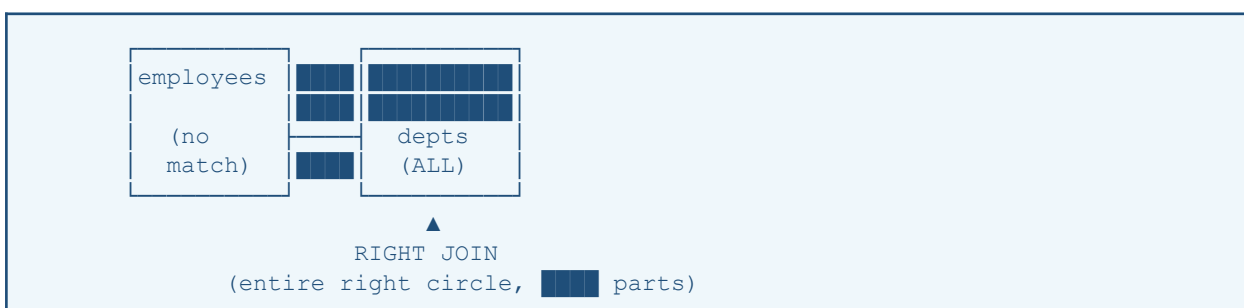
5. RIGHT JOIN (RIGHT OUTER JOIN)

5.1 Concept

RIGHT JOIN is the mirror image of LEFT JOIN. It returns ALL rows from the RIGHT table, and matching rows from the left. Non-matching left rows show NULL.

Think of it as: 'Show me every department, even if no employee is assigned to it.'

5.2 Venn Diagram



5.3 SQL Syntax

```
SELECT e.emp_name, d.dept_name
FROM   employees AS e
RIGHT JOIN departments AS d
      ON e.dept_id = d.dept_id;
```

5.4 Expected Output

emp_name	dept_name
Alice	HR
Diana	HR
Bob	IT
NULL	Finance

5.5 Step-by-Step Explanation

15. Every department is included — no matter what.
16. HR (dept_id=10) → matches Alice and Diana → 2 rows.
17. IT (dept_id=20) → matches Bob → 1 row.
18. Finance (dept_id=40) → NO employee has dept_id=40 → emp_name = NULL.
19. Charlie and Edward do NOT appear — they don't match any existing department.

Key Insight: RIGHT JOIN preserves every row from the right table.

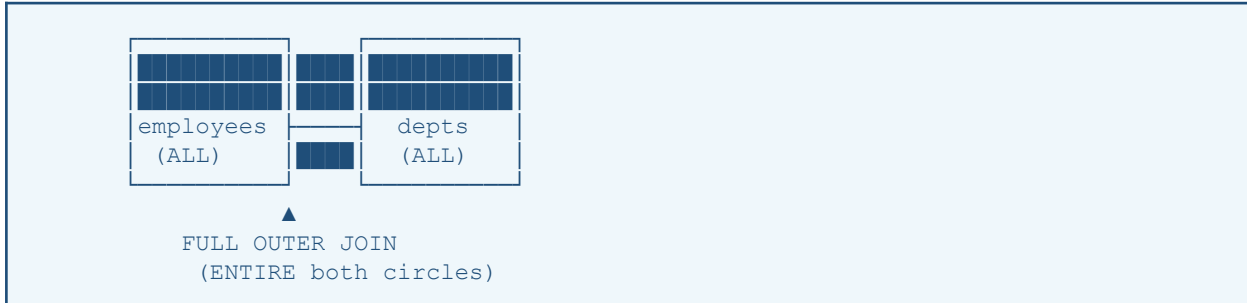
6. FULL OUTER JOIN

6.1 Concept

FULL OUTER JOIN returns ALL rows from BOTH tables. Where there is a match, the columns are filled in. Where there is no match, NULL fills the missing side.

Think of it as: 'I want every employee AND every department — match them where you can, otherwise just show NULL.'

6.2 Venn Diagram



6.3 SQL Syntax

```
SELECT e.emp_name, d.dept_name
FROM   employees AS e
FULL OUTER JOIN departments AS d
      ON e.dept_id = d.dept_id;

-- MySQL doesn't support FULL OUTER JOIN directly.
-- Use this workaround:
SELECT e.emp_name, d.dept_name FROM employees e
      LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name FROM employees e
      RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

6.4 Expected Output

emp_name	dept_name
Alice	HR
Bob	IT
Charlie	NULL
Diana	HR
Edward	NULL
NULL	Finance

6.5 Step-by-Step Explanation

20. All employees appear (like LEFT JOIN).
21. All departments appear (like RIGHT JOIN).
22. Matched rows are combined: Alice↔HR, Bob↔IT, Diana↔HR.
23. Charlie → no dept → dept_name = NULL.
24. Edward → no dept → dept_name = NULL.
25. Finance → no employee → emp_name = NULL.

Key Insight: FULL OUTER JOIN = LEFT JOIN + RIGHT JOIN (minus duplicates).

7. SELF JOIN

7.1 Concept

A SELF JOIN joins a table with itself. This sounds strange, but it's incredibly useful when rows in a table are related to other rows in the SAME table.

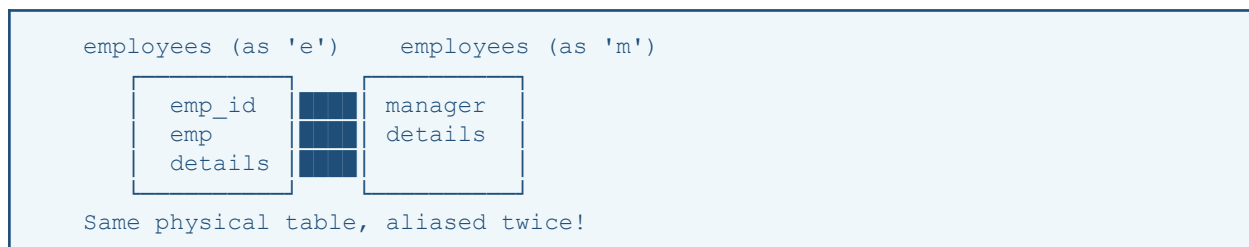
🔑 Classic use case: An employee table where each employee has a manager — and the manager is also an employee in the same table!

7.2 Extended Sample Table for Self Join

We add a `manager_id` column to employees:

emp_id	emp_name	manager_id
1	Alice	3
2	Bob	3
3	Charlie	NULL
4	Diana	1
5	Edward	1

7.3 Venn Diagram



7.4 SQL Syntax

```
SELECT e.emp_name AS Employee,
       m.emp_name AS Manager
FROM   employees AS e
LEFT JOIN employees AS m
      ON e.manager_id = m.emp_id;
```


7.5 Expected Output

Employee	Manager
Alice	Charlie
Bob	Charlie
Charlie	NULL
Diana	Alice
Edward	Alice

7.6 Step-by-Step Explanation

26. We treat employees as two separate virtual tables: 'e' (employee) and 'm' (manager).
27. For each employee in 'e', we look up their manager_id in the 'm' copy of the table.
28. Alice (manager_id=3) → looks up emp_id=3 in 'm' → Charlie.
29. Bob (manager_id=3) → Charlie.
30. Charlie (manager_id=NULL) → no match → Manager = NULL.
31. Diana (manager_id=1) → Alice. Edward (manager_id=1) → Alice.

Key Insight: Aliases (AS e, AS m) are essential — they make SQL treat the same table as two different sources.

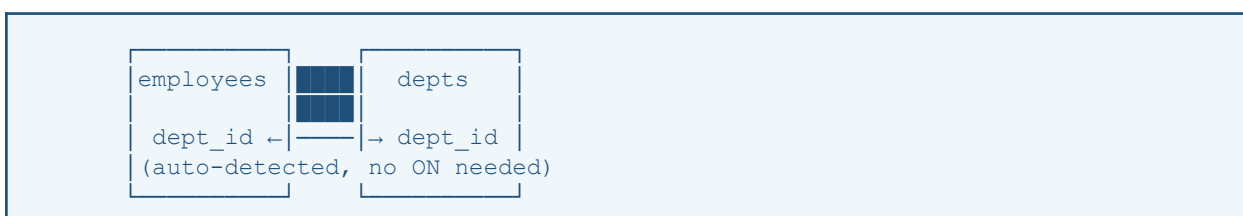
8. NATURAL JOIN

8.1 Concept

A NATURAL JOIN automatically joins two tables on ALL columns that share the same name and data type. You don't write an ON clause — SQL figures it out.

⚠ Warning: Natural Join can be unpredictable if tables have multiple columns with the same name. Use with care!

8.2 Venn Diagram



Acts like INNER JOIN on shared column(s)

8.3 SQL Syntax

```
SELECT emp_name, dept_name
FROM employees
NATURAL JOIN departments;

-- Equivalent to:
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

8.4 Expected Output

emp_name	dept_name
Alice	HR
Bob	IT
Diana	HR

8.5 Key Differences: Natural Join vs INNER JOIN

INNER JOIN	NATURAL JOIN
You specify the join column(s) with ON	Auto-detects shared column names
More control and clarity	Less typing, but less predictable
Recommended for production code	Good for quick queries on well-known tables

9. Quick Reference Cheat Sheet

Join Type	What It Returns	When to Use
INNER JOIN	Only matching rows from both tables	Find records that exist in both tables
LEFT JOIN	All left rows + matched right rows (NULL if no match)	Keep all left records, fill in details from right
RIGHT JOIN	All right rows + matched left rows (NULL if no match)	Keep all right records, fill in details from left
FULL OUTER JOIN	All rows from both tables (NULL where no match)	Find all data including unmatched from either side

SELF JOIN	Table joined with itself via aliases	Hierarchies: managers, parent-child relationships
NATURAL JOIN	Matching rows using auto-detected shared columns	Quick joins on clearly-named shared columns

10. Practice Questions

Using the employees and departments tables from Section 2. Try to solve each question before reading the solution!

BEGINNER LEVEL

Q1: List the names of all employees along with their department names. Show only employees who belong to a department.

 **Hint:** You need only rows where both sides match.

Solution:

```
SELECT e.emp_name, d.dept_name
FROM   employees e
INNER JOIN departments d
      ON e.dept_id = d.dept_id;
```

Expected Output:

emp_name	dept_name
Alice	HR
Bob	IT
Diana	HR

Q2: List ALL employees with their department names. If an employee has no department, show NULL for department name.

 **Hint:** Every employee must appear — use the join type that keeps all rows from the left table.

Solution:

```
SELECT e.emp_name, d.dept_name
FROM   employees e
LEFT JOIN departments d
      ON e.dept_id = d.dept_id;
```

Expected Output:

emp_name	dept_name
Alice	HR
Bob	IT
Charlie	NULL
Diana	HR
Edward	NULL

MODERATE LEVEL

Q3: Show all departments and the number of employees in each. Include departments with ZERO employees.

Hint: Right-side (departments) must always appear. Use COUNT with GROUP BY. Watch out for NULLs in COUNT!

Solution:

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS employee_count
FROM   employees e
RIGHT JOIN departments d
      ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Expected Output:

dept_name	employee_count
Finance	0
HR	2
IT	1

Q4: Display each employee's name along with their manager's name. Employees with no manager should show 'No Manager'.

Hint: The manager is also in the employees table. Use a SELF JOIN with LEFT JOIN + COALESCE or ISNULL.

Solution:

```
SELECT e.emp_name AS Employee,
       COALESCE(m.emp_name, 'No Manager') AS Manager
FROM   employees e
LEFT JOIN employees m
      ON e.manager_id = m.emp_id;
```

```
LEFT JOIN employees m
  ON e.manager_id = m.emp_id;

-- Note: Uses the extended employees table
-- from Section 7 which has the manager_id column.
```

Expected Output:

Employee	Manager
Alice	Charlie
Bob	Charlie
Charlie	No Manager
Diana	Alice
Edward	Alice



Congratulations! You now know all 6 types of SQL Joins.

Practice with real databases to build confidence. The more you query, the more intuitive joins become!

Software Services- Learn To Create